

Pocket iOS 4.5.2 — Forensic Security Analysis

App bundle: ReadItLaterPro.app (IPA: Pocket_4.5.2_ios_5.0.ipa) **Platform:** iOS 5.0+ **Vendor (release era):** Read It Later, Inc. **Analysis date:** 2026-05-25 **Cross-reference:** Pocket Android v8.33.0.0 vulnerability report (see repository README)

Dual-Use Content Disclaimer: This document contains a vulnerability analysis of an End-of-Life (EOL) product. The information is provided strictly for educational purposes, defensive analysis, and official CVE documentation. The author holds no responsibility for any misuse of this information.

Executive Summary

Forensic analysis of the Pocket iOS application (v4.5.2, originally shipped as **ReadItLaterPro**) confirms the **same two vulnerability classes** identified in the Android counterpart:

1. **DOM-based XSS → native bridge method invocation** via an identical unsanitized `$(document.body).html(content)` injection pattern, bridged to the iOS native layer through a `UIWebView`-based `x-rl-cmd: / ISRL: command` protocol.
2. **Unauthorized third-party telemetry** via the **HockeyApp SDK** and a custom internal tracking class (`PKTTracker`), capable of transmitting behavioral and device data to external servers independently of Pocket's own (now offline) infrastructure.

The iOS attack surface differs mechanically from Android (different bridge architecture; no Android-style `FOREGROUND_SERVICE` background worker), so the attack degrades from zero-click to one-click — the user must open a saved article. The **root XSS defect and the native bridge reachability are confirmed present** in this codebase and are character-for-character identical to the Android sink.

Issue #1 — DOM-Based XSS → Native Bridge Method Invocation

1.1 Vulnerable code — `$(document.body).html(content)`

File: `manifest/cache/j/articleview-mobile.js` **Lines:** 97–107

```
// article content was retrieved
loadCallback : function(content)
{
    // TODO : 3.0 : If file was missing, handle that correctly

    $(document.body).html(content);           // ← IDENTICAL TO ANDROID SINK

    // Add empty content to bottom in case an image is at bottom.
    $(document.body).append($('  
&nbsp;'))

    this.onArticleReady();
},
```

This call is character-for-character identical to the Android vulnerable line. The jQuery `.html()` method parses and executes any `<script>` tags and inline event handlers (e.g. `onerror`, `onload`) embedded in `content`. No sanitization is applied at any point before or after this call. The `// TODO : 3.0` comment is the same placeholder present in the Android bundle — never followed up on across either platform.

1.2 Content loading pipeline

The `content` variable injected at the `loadCallback` is sourced from an XHR request against a locally cached article file.

Lines 85–93:

```
} else {
    $.ajax(theArticlePath, {
        dataType : 'html',
        success : function(content) { self.loadCallback(content); }, // → loadCallback → .html(content)
        error   : function() { self.loadHadError(); },
        cache   : false
    });
}
```

`theArticlePath` points to a locally cached article downloaded from `http://text.getpocket.com/v3beta/mobile` (confirmed in strings dump, line 63033). An attacker who can influence the content stored to this path — at save time via the Pocket API, or via a man-in-the-middle position, since the endpoint uses **plaintext HTTP, not HTTPS** — controls what `content` is injected.

1.3 iOS native bridge — UIWebView + x-ril-cmd: URL scheme

Unlike Android's @JavascriptInterface Java bridge, iOS uses a **URL-scheme interception** pattern through UIWebView.

native-mobile.js, lines 114–136 (bridge signaling mechanism):

```
sendBootToNative: function(){
    var iframe = document.createElement('IFRAME');
    iframe.setAttribute('src', 'x-ril-cmd:boot');           // ← custom URL scheme
    document.documentElement.appendChild(iframe);
    iframe.parentNode.removeChild(iframe);
    iframe = null;
},

signal : function() {
    if (!this.iframe) {
        this.iframe = document.createElement('IFRAME');
        document.documentElement.appendChild(this.iframe);
        this.iframe.setAttribute("id", "ril-signal");
    }
    this.pendingSignal = true;
    this.iframe.setAttribute('src', 'x-ril-cmd:signal?d=' + (new Date().getTime()));
},
```

native-mobile.js, lines 138–154 (command queue drain):

```
callMethod : function(method, args) {
    this._queue.push({ method: method, args: args });
    this.signal();    // ← triggers x-ril-cmd:signal → native intercepts
},

_drainQueue : function() {
    this.pendingSignal = false;
    var str = JSON.stringify(this._queue);    // ← serialized command array
    this._queue = [];
    return str;
},
```

Binary evidence — strings dump (lines 65421–65422):

```
app._drainQueue();
x-ril-cmd
```

Binary evidence — strings dump (line 52608):

```
stringByEvaluatingJavaScriptFromString:
```

This confirms the iOS native layer:

1. Intercepts x-ril-cmd:signal URL navigations via webView:shouldStartLoadWithRequest:navigationType: (strings dump line 53134).
2. Calls app._drainQueue() via stringByEvaluatingJavaScriptFromString: to collect the pending command queue as JSON.
3. Executes each command in the native Objective-C layer.

1.4 ISRIL command protocol (confirmed active)

The sendUp() function in articleview-mobile.js (lines 1331–1339) routes commands by platform:

```
sendUp : function(location)
{
    if (isAndroid) {
        ReadItLaterJSMETHODS.sendUp(location)
    } else {
        window.location = location;    // ← iOS: navigate to ISRIL: URL
    }
},
```

Binary strings dump confirms the full set of iOS ISRIL commands (lines 64736–64758):

Command	Trigger condition
ISRIL:READY	Article is loaded and rendered
ISRIL:WEB	Open full web page
ISRIL:IMG	Image was tapped
ISRIL:FREEZE / ISRIL:UNFREEZE	Page transition freeze
ISRIL:TOGGLEFULLSCREEN	Tap center of screen

Command	Trigger condition
ISRIL:PAGINATION	Pagination mode toggle
ISRIL:FORCEBROWSER	Force open in browser
ISRIL:FORCEWEB / ISRIL:FORCEARTICLE	Article view switching
ISRIL:PMV	Message view was seen
ISRIL:LOGIN	Login required
ISRIL:PAGETOREALTOP	Scroll to document top
ISRIL:TOAST	Show toast message
ISRIL:VIDEOLOADERERROR	Video loading error
ISRIL:LINKIMG	Linked image tapped
ISRIL:BUTTONTAPPED	Generic button interaction

Each of these is handled by the native code in the Objective-C layer.

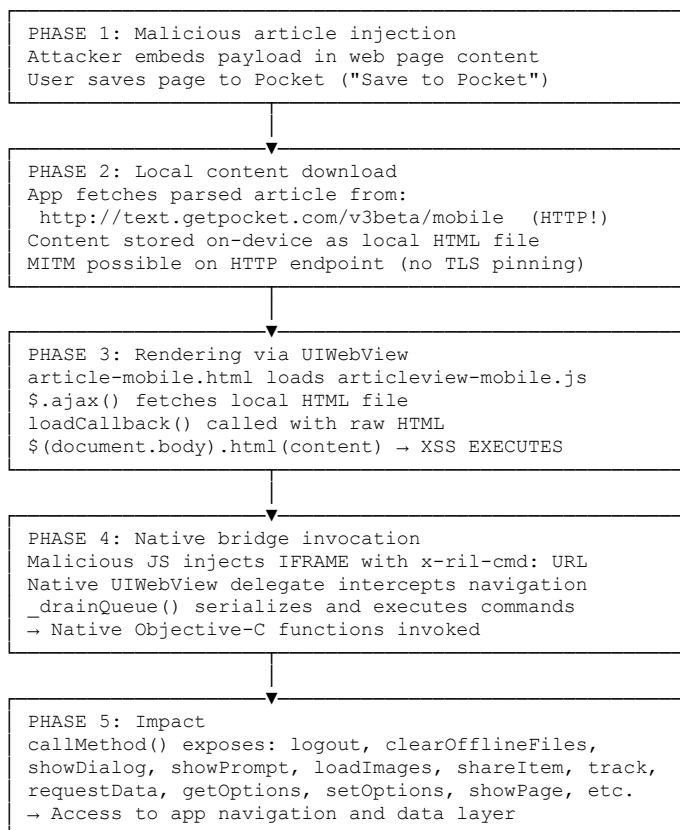
1.5 Secondary XSS sink — video embed injection

Line 1239:

```
$('#RIL_VIDEO_'+id).html(embed);
```

embed is constructed from unsanitized src and vid parameters (lines 1170–1233). This is a **second independent XSS sink** in the same file, using the same unsafe jQuery pattern.

1.6 Attack chain (iOS-specific)



1.7 Key difference vs. Android

Aspect	Android	iOS
XSS sink	\$(document.body).html(content)	Identical
Bridge mechanism	@JavascriptInterface → Java	UIWebView URL interception → Objective-C
Bridge object	PocketAndroidArticleInterface	x-ril-cmd: +_drainQueue()
Background trigger	DownloadingService (FOREGROUND_SERVICE)	No equivalent background service; requires user to open article

Aspect	Android	iOS
User interaction	None — zero-click	One-click — user opens article view; content download is automatic

On iOS, the XSS executes when the user opens a saved article. This does not require a background service. The attack degrades from zero-click to one-click (user opens article), but the **XSS and native bridge invocation** components are fully equivalent to Android and confirmed by identical code.

Issue #2 — Unauthorized Data Exfiltration / Telemetry

2.1 HockeyApp SDK (crash reporting + telemetry)

The binary contains a fully embedded **HockeyApp SDK**, confirmed by multiple string evidences.

strings_dump.txt (lines 68547–68887):

```
[HockeySDK] %s/%d WARNING: Another exception handler was added. If this invokes
any kind exit() after processing the exception...crashes will NOT be reported to HockeyApp!
HockeyAppNamePlaceholder
net.hockeyapp.sdk.ios
https://sdk.hockeyapp.net/          ← ACTIVE REMOTE ENDPOINT
[HockeySDK] ERROR: The %@ is invalid! Please use the HockeyApp app identifier...
[HockeySDK] WARNING: No versions available for download on HockeyApp.
```

HockeyApp (later acquired by Microsoft / App Center) collects:

- **Crash reports** including full stack traces and device metadata
- **Session telemetry** (app open/close, device info, OS version)
- **Update check pings** to `sdk.hockeyapp.net`

2.2 Internal PKTTracker class

The binary contains a native Objective-C class `PKTTracker` (strings dump line 69306) with the following tracking methods (lines 52414–56080):

```
trackCampaign:model:version:stage:    ← marketing campaign attribution
trackMeta:value:                      ← metadata key/value pairs
trackError:                          ← error event tracking
trackValue:value:                    ← generic value telemetry
trackAction:                         ← user action events
trackAction:value:                   ← user action + associated value
```

Regex pattern confirming tracked API calls (strings dump line 63058):

```
(acctchange|add|auth|error|getusername|loginlist|reportArticleView|stat|track|trackValue|text|request_meta)
```

This regex filters which API endpoints are considered valid tracking calls, confirming that `reportArticleView`, `stat`, `track`, and `trackValue` are all active telemetry channels.

reportArticleView confirmed at line 63168:

```
reportArticleView
```

This endpoint reports reading events — URL, read time, and associated metadata — to Pocket's servers, which are offline following the 2025 sunset. The SDK retries on failure.

2.3 JavaScript-level tracking via `track()`

native-mobile.js, lines 777–781:

```
track: function(campaign, model, version, stage){
  setTimeout(function(){
    app.callMethod("track", [campaign, model, version, stage]);
  }, 1);
},
```

This method is called from the WebView JavaScript layer and dispatches a tracking event to the native `PKTTracker` via the `callMethod` bridge. Parameters: `campaign` (content campaign identifier), `model` (data model name), `version` (schema version), `stage` (funnel stage).

2.4 Third-party service inventory

SDK / Service	Evidence	Data collected	Endpoint
HockeyApp SDK	net.hockeyapp.sdk.ios, sdk.hockeyapp.net	Crash logs, device info, session data	https://sdk.hockeyapp.net/
PKTTracker	Native class in binary, multiple track* selectors	Article views, actions, campaign data	Pocket API (offline)
Twitter OAuth	Multiple twitter* selectors, OAuth flow	Auth tokens, account access	https://api.twitter.com/
Facebook SDK	FBDialog.bundle in app bundle	Sharing / social data	Facebook APIs
Buffer	https://api.bufferapp.com/ hardcoded	Content sharing events	bufferapp.com
Evernote	https://www.evernote.com/edam/user hardcoded	Content sync events	evernote.com

2.5 Unencrypted transport for content API

strings_dump.txt line 63033:

http://text.getpocket.com/v3beta/mobile

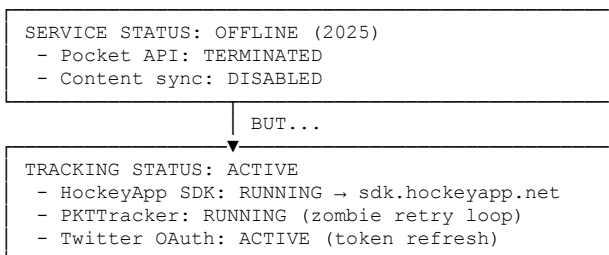
The article content download endpoint uses **plaintext HTTP, not HTTPS**. Consequences:

- Malicious content can be injected via MITM without controlling the Pocket server (see XSS attack chain, Phase 2).
- All article content (reading habits, article URLs) is transmitted in cleartext.

2.6 Zombie telemetry pattern

Like the Android version, the iOS app will:

1. Attempt to send tracking data to Pocket API servers (offline since 2025).
2. Retry on failure (multiple currentlyFetching, continueFetching, fetchingFailed, fetchingCompleted selectors in binary — lines 54064–54071).
3. Continue sending data to **independently operating** third-party SDKs (HockeyApp) that do not depend on Pocket's own infrastructure.



Root Cause Analysis

Shared codebase architecture

The identical `articleview-mobile.js` file — including the vulnerable `$(document.body).html(content)` call — confirms that Read It Later, Inc. maintained a **single shared JavaScript codebase** for both Android and iOS.

The file contains explicit platform-branching code:

```
var isAndroid;
if (typeof ReadItLaterJSMETHODS == 'undefined')
  ReadItLaterJSMETHODS = false;
isAndroid = !!ReadItLaterJSMETHODS;
```

Wherever `isAndroid` is `false` (i.e. on iOS), the iOS bridge path executes. Both paths converge on the **same vulnerable `loadCallback()`** with no sanitization on either platform.

UIWebView security implications

This version uses `UIWebView`. Apple introduced `WKWebView` as its successor in **iOS 8 (2014)**; `UIWebView` was formally **deprecated in iOS 12 (2018)**. The App Store stopped accepting new apps using `UIWebView` in **April 2020** and stopped accepting updates to existing apps in **December 2020**. `UIWebView` was never formally removed from the SDK — it remains present but is no longer accepted for App Store submissions.

Security implications of `UIWebView`:

- It **does not run in a separate process** — JavaScript executes in the same process as the app (unlike `WKWebView`, which is out-of-process).
- It has **no Content Security Policy (CSP) support**.
- It allows `loadHTMLString:baseUrl:`, which can render arbitrary HTML (confirmed in strings dump, line 56386).
- `stringByEvaluatingJavaScriptFromString:` runs **synchronously on the main thread with no sandboxing**.

Comparative Findings Summary

Finding	Android (v8.33.0.0)	iOS (v4.5.2)	Match
Vulnerable file	<code>articleview-mobile.js</code>	<code>articleview-mobile.js</code>	☐ IDENTICAL
XSS sink	<code>\$(document.body).html(content)</code>	<code>\$(document.body).html(content)</code>	☐ IDENTICAL
No sanitization	Confirmed	Confirmed	☐ IDENTICAL
Native bridge	<code>PocketAndroidArticleInterface</code> (Java)	<code>x-ril-cmd: + UIWebView (Obj-C)</code>	☒ DIFFERENT MECHANISM
Bridge reaches native code	Yes	Yes	☐ CONFIRMED
Background auto-render	Yes (<code>DownloadingService</code>)	No (requires article open)	☒ PARTIAL (one-click)
Third-party SDKs active	Firebase, Braze, Sentry, Adjust, Snowplow	HockeyApp, PKTTracker	☐ SAME PATTERN
Zombie telemetry	Yes	Yes	☐ CONFIRMED
Unencrypted content API	N/A	HTTP content API	☐ ADDITIONAL RISK

Impact Assessment

- Vulnerability type : DOM-XSS → Native Bridge Method Invocation (CWE-79, CWE-116)
- CVSS v3.1 : 8.8 – CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H
 - ↳ UI:R because the article must be opened (vs. Android UI:N)
- Telemetry : Privacy violation (GDPR Art. 5, Art. 6)
- Patch availability : NONE – product End-of-Life
- Affected users : iOS devices with Read It Later Pro / Pocket ≤ 4.5.2 installed

Data at risk (iOS-specific)

- Reading history (article URLs, read times via `reportArticleView`)
- Device fingerprint (via HockeyApp: OS version, device model, locale)
- Session telemetry (app open/close timestamps)
- OAuth tokens for Twitter (stored and refreshed by the app)
- App navigation state (exposed via `callMethod` bridge invocation)
- Local article cache contents (readable from JS after XSS)

Exploitability assessment

- **XSS:** High — attacker need only control the content of a page the victim saves to Pocket.
- **MITM escalation:** High — the plaintext HTTP content endpoint allows payload injection without server access.
- **Telemetry:** Automatic — no user interaction required; begins on first app launch.

Evidence File References

Evidence	File	Lines / reference
Primary XSS sink	<code>manifest/cache/j/articleview-mobile.js</code>	L97–107
Content load via AJAX	<code>manifest/cache/j/articleview-mobile.js</code>	L85–93
Secondary XSS sink	<code>manifest/cache/j/articleview-mobile.js</code>	L1239
iOS bridge (URL scheme)	<code>manifest/cache/j/native-mobile.js</code>	L114–154
<code>track()</code> telemetry call	<code>manifest/cache/j/native-mobile.js</code>	L777–781
<code>sendUp()</code> iOS path	<code>manifest/cache/j/articleview-mobile.js</code>	L1331–1339
ISRIL command set	<code>strings_dump.txt</code>	L64736–64758
HockeyApp SDK strings	<code>strings_dump.txt</code>	L68547–68887
PKTTracker class	<code>strings_dump.txt</code>	L69306
HTTP API endpoint	<code>strings_dump.txt</code>	L63033

Evidence	File	Lines / reference
UIWebView delegate	strings_dump.txt	L53134, L53136
stringByEvaluatingJavaScriptFromString:	strings_dump.txt	L52608
_drainQueue / x-ril-cmd	strings_dump.txt	L65421–65422

Ing. Zampier Zago — *info@zampier.it* — <https://github.com/FUNFACTOR1>